

Introduction

Feedforward neural networks (FNNs) are one of the simplest types of artificial network, and they serve as the foundational structure for many of the complex models seen today. FNNs are defined by a one-way flow of information through hidden layers, often used for pattern recognition tasks. An FNN consists of an input layer, zero or more hidden layers, and an output layer through which data propagates. The input layer is made of neurons that receive raw data, and its size is determined by the data dimensions. The hidden layers are essentially the network's "computational engine." Each neuron in the hidden layers computes a weighted sum of the outputs from the previous layer, applies an activation function, and passes this result to the next layer. The weights symbolize the importance of a particular input, and the activation function decides if the neuron "fires" or not. This enables the network to identify relationships and patterns in the data. Finally, the output layer provides the network's prediction, with its size matching the desired number of outputs.

The mathematical basis for FNNs was inspired by biological neurons in the brain, which receive and propagate signals between each other, deciding whether they should fire. The first model of a "neuron" was a single layer network that was able to perform binary classification. This simple network, known as the Perceptron, could only solve linearly separable problems, but it laid the groundwork for modern FNNs which can use nonlinear activation functions to classify more complex data (such as Tanh or ReLU). Today, there are more accurate and efficient types of neural networks, but FNNs are still used in several areas, mainly pattern recognition tasks. For example, a bank estimating a client's creditworthiness based on factors such as credit history and spending habits. This paper will examine how FNNs are able to perform tasks like this by looking at their structure as well as different preprocessing methods, activation functions, and training algorithms. It will also highlight this network's shortcomings when it comes to classification tasks.

Implementation

To analyze the properties of FNNs, we wrote a MATLAB script to classify images in the CIFAR-100 dataset. This dataset contains 100 classes with 600 images each, which are split up into 500 training images and 100 testing images. These 100 classes are grouped into 20 "superclasses," which is what our FNN is focused on classifying. CIFAR-100 gives images that are 32 x 32 pixels with 3 color channels each. The layout of the dataset is described on the CIFAR website, but in summary, the images are stored in a 3072 x 50000 array where each row is a feature across all 50000 images.

The bulk of our FNN comes from MATLAB's `patternnet` function, which provides the foundation for a basic pattern recognition FNN. This function allows the user to specify the number and size of hidden layers, as well as the training algorithm to use. The hidden layers automatically use the hyperbolic tangent activation function, but this can be changed outside of `patternnet`. The training algorithm optimizes the weights in the hidden layers based on the value of a loss function. For

patternnet, the default function to calculate loss is cross-entropy. The output layer gives class probabilities (20 classes in our case), with the sum of all the outputs equaling one. By changing the training algorithm, activation function, and method of preprocessing, we were able to find a combination that most accurately classified the CIFAR-100 dataset.

Preprocessing

Using principal component analysis (PCA), we preprocessed our data in order to increase our classification accuracy and decrease training time. PCA is a dimension reduction method commonly used in algorithms to reduce noise and overfitting in data. It works by using singular value decomposition on the feature array A to get it into the form:

$$A = USV^T$$

Where U and V contain normalized eigenvectors of $A^T A$ and AA^T and S is a diagonal matrix with entries decreasing down the diagonal. The values of S come from the singular values of A . Each entry measures how much of the data's total variance occurs along the i th principal direction. The rank of A is equal to the rank of S , and since S is a diagonal matrix, we can truncate matrix S by replacing entries after a certain column (k) with zero. This gives the approximation:

$$A \cong U_k S_k V_k^T$$

In turn, the dimension of matrix A is reduced while keeping most of the total variance information. In our script, we implemented PCA by calling MATLAB's built in `pca` function and setting k equal to 550 which captured about 98.5% of the variance. This is visualized in Figure 1.

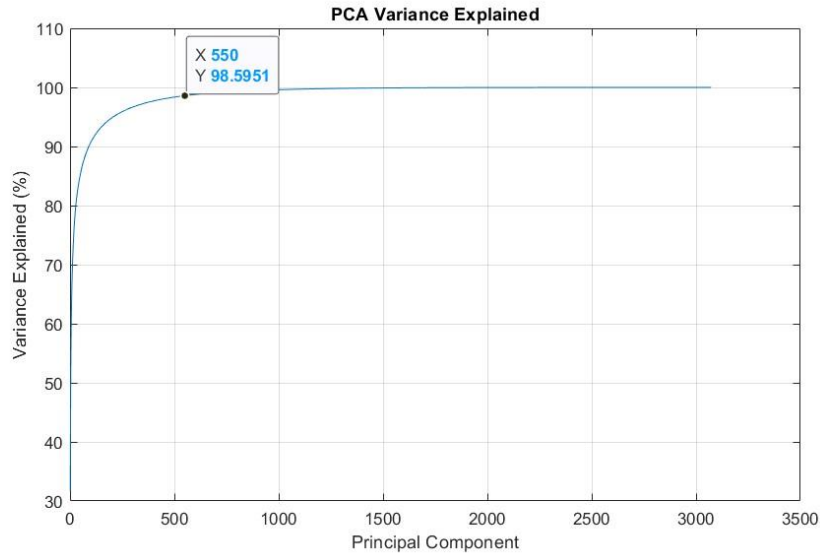


Figure 1: Plot of PCA Variance

Activation Functions

An activation function is used to determine whether a neuron should fire based on the input it receives. We tested two different activation functions for our FNN: Tanh and ReLU. The hyperbolic tangent function (Tanh) is the default activation function for patternnet and is defined by the following expression:

$$\text{Tanh}(x) = \frac{2}{1 + e^{-2x}} - 1$$

ReLU is a more widely used and computationally efficient activation function, and is defined by:

$$\text{ReLU}(x) = \max(0, x)$$

Cost Function

We used the default cost function for patternnet, which was cross entropy. This function is defined by the formula:

$$J = - \sum_{i=1}^k y_i \log(p_i)$$

Here, J is the cost, y_i is the true label (a vector of all 0's and a 1 for the correct class), and p_i is the predicted probability of the class i after using the activation function. If y_i is close to p_i then the loss will be small, meaning our FNN is working accurately.

After this cost function is calculated via back-propagation, the weights and biases at each node are updated to minimize the previous cost for each epoch. This minimization is done using our specified training algorithm.

Training Algorithms

Using the values from the cost function described above, we tested two similar training algorithms to update our weights and biases: gradient descent (GD) and conjugate gradient descent (CGD). The mathematical differences between the two are summarized in Table 1.

GD is a mathematical model that seeks to lower the gradient of the cost function during each epoch. This is done by computing J on the current weights, back-propagating to compute ∇J at those weights, and updating each weight by subtracting $\eta \nabla J$ from its current value, where η is a chosen learning rate ($\eta = 0.01$ in our case). This process is done for each weight in each layer. For gradients greater than zero, loss is increasing, and thus, we would want to decrease the weight of the associated node. By subtracting this gradient from the weight, we make it smaller. For gradients

less than zero, the opposite is true, and we end up making the weight of the node larger. Overtime, this leads to a more accurate FNN where neurons that lower the cost function are given more importance. The same idea is applied to the biases as well. Due to the amount of computation in GD, it can take a long time to train a network, especially when there are many neurons and hidden layers. To implement this algorithm in MATLAB, we used ‘traingd’ in patternnet.

CGD is similar to gradient descent, but instead of using a fixed learning rate η , it uses a parameter α_k that combines the current gradient with the previous step direction (parameter β_{k-1}). Its movements are conjugate (orthogonal) to previous directions, and in turn, progress from earlier movements is never undone. This “smarter” way of searching for a lower cost makes CGD fast and more accurate than GD.

Algorithm	Search Direction	Step-Size	Update
GD	$p_k = -\nabla J(w_k)$	Fixed η	$w_{k+1} = w_k + \eta p_k$
CGD	$p_k = -\nabla J(w_k) + \beta_{k-1} p_{k-1}$	α_k chosen by line search	$w_{k+1} = w_k + \alpha_k p_k$

Table 1: Comparison between GD and CGD training algorithms

Method

Our method of data collection for Table 2 went as follows:

1. Set patternnet to contain two hidden layers with 200 neurons each
2. Compare GD and CGD with and without PCA preprocessing using the Tanh function
3. Using the accuracy, determine the better training algorithm
4. Use the more accurate training algorithm to compare ReLU and Tanh functions with and without PCA
5. Determine the overall best combination

Results

After running several combinations of preprocessing, activation functions, and training algorithms, we compared their accuracies seen in Table 2. The accuracies were calculated using the following approach, with $n = 10000$ (for the 10000 test images).

$$Accuracy \% = \left[\frac{1}{n} \sum_{i=1}^n (predicted(i) = true(i)) \right] * 100$$

Training Algorithm	PCA Preprocessing	Activation Function	Accuracy (%)
GD	No	Tanh	17.20
CGD	No	Tanh	31.10
GD	Yes	Tanh	5.10
CGD	Yes	Tanh	27.13
CGD	Yes	ReLU	16.86
CGD	No	ReLU	26.18

Table 2: Accuracies of various combinations

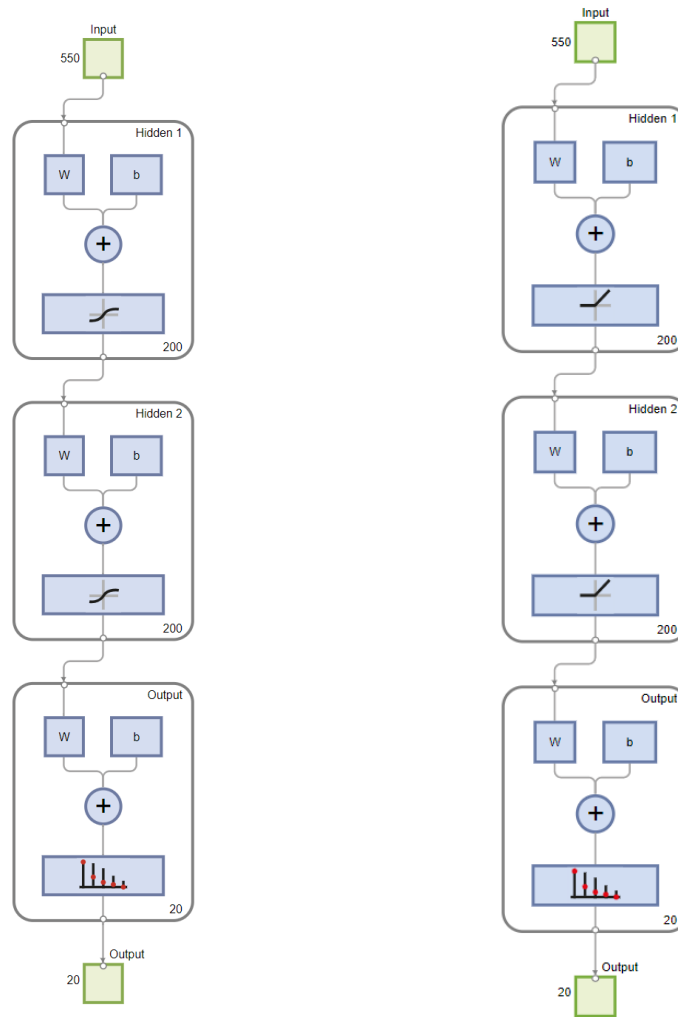


Figure 2: Structure of FNN, Tanh vs ReLU

Figure 2 shows the structure of our FNN. Both diagrams have used PCA for the input layer. The left diagram shows a Tanh activation function in the hidden layers while the right diagram shows ReLU.

Discussion

Overall, Table 2 shows several expected patterns, as well as many results that do not quite match our research. CGD proved to be better and faster than GD, which was expected. However, the Tanh activation function seemed to outperform ReLU, and PCA did not improve our accuracy.

Training Algorithm: GD vs CGD

As predicted, CGD consistently outperformed GD in both accuracy and training time. For example, without preprocessing and using the Tanh activation function, GD achieved only 17.2% accuracy, while CGD achieved 31.1% (Table 2). CGD also typically completed training in under 30 minutes, much faster than GD which often took over an hour. These results highlight the more efficient method of CGD, which has a more advanced optimization technique that improves upon basic gradient descent by using information about previous gradients to adjust the search direction more intelligently. Instead of always stepping directly downhill like GD, CGD uses conjugate directions that allow it to converge more quickly. Subsequently, it reduces the number of training iterations needed and often finds better minima, leading to significantly higher classification accuracy.

Preprocessing: PCA On vs. Off

Preprocessing the input data with Principal Component Analysis (PCA) was intended to reduce noise and overfitting; however, in this experiment, applying PCA actually reduced classification performance. When PCA was used to reduce the input dimension to 550 principal components (see Figure 1), the network's accuracy decreased across the board. For instance, with GD and Tanh activation, the accuracy dropped from 17.2% to 5.1% after adding PCA. A similar drop was seen for CGD with Tanh activation, from 31.1% to 27.13%. This decline can be explained by the fact that CIFAR-100 is a highly complex data set, and even a small reduction in feature space can cause important information necessary for classification to be lost. Although PCA preserves most of the variance, it does not guarantee preservation of the features that are most relevant for separating classes. Therefore, reducing the input dimensions made it harder for the FNN to correctly distinguish between the 20 superclasses, leading to worse performance.

Activation Function: Tanh vs. ReLU

Two activation functions, hyperbolic tangent (sigmoid) and ReLU, were compared to assess their impact on classification performance. Contrary to our research, the Tanh activation function performed better than ReLU. Without preprocessing, using CGD and Tanh achieved 31.1% accuracy, compared to 26.18% for ReLU. Similarly, with PCA preprocessing, Tanh achieved 27.13% while ReLU only had 16.86%. Although ReLU is widely considered more efficient and effective in deep networks, as it avoids the vanishing gradient problem, this advantage is less noticeable in shallow networks like the one we built using MATLAB's patternnet. Our network had 200 neurons per hidden layer, but it only had 2 of these layers. In shallow networks, sigmoid or Tanh activation functions can sometimes provide better separation between classes, especially

when the input data is already normalized or relatively small in scale. Additionally, ReLU has a known issue where neurons can "die" during training (outputting zero permanently), which can hurt performance when there are fewer layers and fewer neurons available, as is typical with simple FNNs. In this case, the Tanh activation allowed the FNN to learn more nuanced decision boundaries, resulting in better classification accuracy.

Overall Best Configuration

The best performance, with an accuracy of 31.1%, was achieved using the CGD training algorithm, no preprocessing (raw data input), and the Tanh activation function. This configuration allowed the network to fully utilize all available image information, converge efficiently during training, and develop more effective boundaries between classes. The combination of a powerful optimizer, access to the full feature space, and a smoother activation function worked together to maximize the FNN's performance on the CIFAR-100 superclass task.

Note about FNNs for Image Classification

As seen in Table 2, none of the classification accuracies were very high. This is because FNNs are generally not very good at complex image classification. This fact arises from their large number of parameters and disregard of spatial structure. The number of parameters can quickly lead to overfitting on large datasets, which we saw on several of our training runs. One of these overfitting cases is illustrated in Figure 3 below.

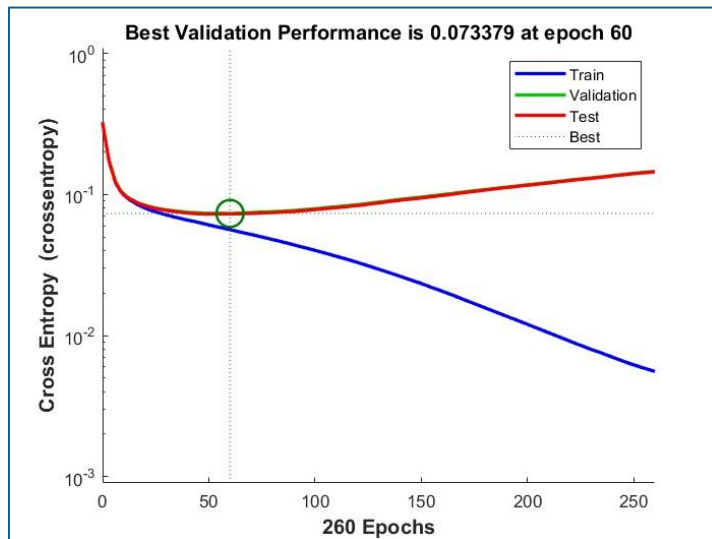


Figure 3: Loss vs Epoch plot shows overfitting

The large gap between testing and training shows the model is overfitting. Before epoch 60, generalization is being improved as all losses are decreasing. However, after epoch 60, the model begins “memorizing” the training data. The training loss drops rapidly, but the test and validation losses start increasing. This means the model is learning the noise in the training data at the expense of generalization. In other words, it is overfitting.

Although FNNs are not ideal for complex image classification, there are many neural networks that can classify images from the CIFAR-100 dataset with close to 100% accuracy. One such example is explained below.

CNN vs. FNN for Image Classification

When working with images, Convolutional Neural Networks (CNNs) are typically a much better choice than regular Feedforward Neural Network. Firstly, CNNs are designed to take advantage of how images work. In an image, nearby pixels are related or grouped together. CNNs look at small patches of an image at a time, instead of trying to connect every single pixel to every neuron like FNNs do. This makes CNNs way more efficient and means they need far fewer parameters.

Why fewer parameters matter:

Fewer parameters mean CNNs are faster to train, take up less memory, and don't overfit the training data as easily. In comparison, FNNs have to learn a different weight for every single pixel-to-neuron connection, which wastes a lot of effort and can cause overfitting.

Weight sharing:

CNNs also use the same small set of weights, which is commonly referred to as a filter, across the whole image. The filter goes through the image, looking for patterns like edges or textures in its patches. FNNs don't share weights, rather they treat every part of the image separately meaning they can't easily recognize the same pattern if it appears somewhere else. When CNNs learn these patterns, they can then combine those into bigger patterns in deeper layers to start identifying entire objects. FNNs can't do this as easily because they don't quite grasp the structure of images.

Repeated images:

CNN's have a special property where if an image with objects or animals appears multiple times, the CNN can remember these patterns and realize what that object or animal is. However, a FNN must relearn this object independently every time.

Testing results:

In tests on this data set, CNNs perform way better than FNNs. A CNN can reach decent accuracy of anywhere from 50 – 90%, while a FNN on the same images that has been modified and trained is only reaching 30%. A completely basic FNN gets around 1-3% accuracy which is near what we saw with our test run (Table 2), but we found that a completely basic CNN can get around 24% on the CIFAR-100, from a GITHUB report.

Conclusion

In this project, we built and evaluated a two-hidden layer feedforward neural network (FNN) using MATLAB's pattenet to classify CIFAR-100 superclasses. In doing so, we explored the effects of preprocessing using PCA, activation functions (Tanh vs. ReLU), and optimization algorithms (GD vs. CGD). Our results showed that CGD with a hyperbolic tangent activation function and no PCA achieved the highest accuracy (31.1%) on our CIFAR dataset. This combination outperformed regular GD by almost 14 percentage points and ReLU by about 5 points.

We found that reducing dimensionality with PCA hindered performance in every case, implying that even small losses in variance can cause critical information to be ignored when working with complex data. We also saw that the smooth, zero-centered Tanh activation function was more effective in classifying images compared to the piecewise linear function used in ReLU. It avoided dead neurons and helped maintain the gradient. Lastly, the conjugate search method in CGD led to faster convergence and better minima than GD did. These facts contributed to our final and best configuration.

Despite finding the best combination among our different alterations to the FNN, we still only achieved a maximum accuracy of 31.1%. This is well below the accuracy levels that could be achieved by convolutional neural networks (CNNs), highlighting the limitations of FNNs for complex image classification. These limitations include massive parameter counts and disregard for special features, which lead to overfitting and inefficient feature learning. A future project would focus on developing a CNN and testing other training techniques, data processing, and activation functions to see if we could more accurately classify the CIFAR-100 dataset.

Works Cited

- CIFAR-10 and CIFAR-100 datasets*. (n.d.). <https://www.cs.toronto.edu/~kriz/cifar.html>
- DeepAI. (2020, June 25). *Feed Forward neural network*. DeepAI. <https://deepai.org/machine-learning-glossary-and-terms/feed-forward-neural-network>
- What Is a Neural Network and its Types?*-Spiceworks Inc. (2025, March 10). Spiceworks Inc. <https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-a-neural-network/>
- Kim, S. (2021). Machine Learning CW4 : Report. In *GITHUB*. Ulsan National Institute of Science and Technology (UNIST). Retrieved April 26, 2025, from https://github.com/solangii/CIFAR10-CIFAR100/blob/master/report/20181041_SolangKim_Report.pdf
- GeeksforGeeks. (2025, February 3). *Principal Component Analysis(PCA)*. GeeksforGeeks. <https://www.geeksforgeeks.org/principal-component-analysis-pca/>
- GeeksforGeeks. (2025b, April 5). *Activation functions in Neural Networks*. GeeksforGeeks. <https://www.geeksforgeeks.org/activation-functions-neural-networks/>